COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/16/2026
NAME(s):	Larracas, Jerry Matthew L. Quiming, Kadaphy James A.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A

Strategy	Algorithm	Analysis
Recursion	Recursive DFS	[Larracas] It is a function that has a loop that stops until it meets its condition. Sequential search is one of the example of recursive because it searches values to display them in order until it reaches all of the elements which is the stopping point. [Quiming] A function calls itself until it reaches a stopping point.
Brute Force	Linear search	[Larracas] This algorithm searches elements one by one linearly until the if finds the specific element that is set to find. [Quiming] Searches every element one by one until it finds the target element, it tries every possible position without shortcuts
Backtracking	Depth first search	[Larracas] The idea is to do a depth search by visiting all of the branches or path as deeply as possible before backtracking to try another path or branch. [Quiming] Just like the previous activity backtracking is just like DFS which goes down one path as deep as possible before backing up
Greedy	Selection sort	[Larracas] It chooses the best available option at each step to reach a solution efficiently. [Quiming]

		Choose the best option at each step.
Divide-and-Conquer	Merge sort	[Larracas] It divides or halves a the probleminto smaller sub-problems, then solve them separately and combines their results to solve the original problem. [Quiming] Splits a problem into smaller parts, solves them, and combines the results.

Memoization Implementation

ILOA.cpp > getMinSteps(int)

```cpp
// Activity 12 - ILO A: Minimum Steps to one (Memoization / Top-Down)
#include <iostream>
#include <vector>
#include <algorithm>

std::vector<int> memo;

int getMinSteps(int n){
    if ( n == 1 ) return 0; // base case
    if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the optimal answer finally
    if( n%2 == 0 ) r = std::min( r , 1 + getMinSteps( n / 2 ) ) ; // '/2' step
    if( n%3 == 0 ) r = std::min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}

int main(){
    int test[] = {1, 4, 7, 10, 15, 100};

    for(int n: test){
        memo.assign(n + 1, -1);
        std::cout << "n = " << n << " -> Minimum Steps = " << getMinSteps(n) << std::endl;
    }

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS D:\_DOWNLOADS\DO_THIS\HOA 12> cd "d:\_DOWNLOADS\DO_THIS\HOA 12\" ; if ($?) { g++ ILOA.cpp -o
n = 1 -> Minimum Steps = 0
n = 4 -> Minimum Steps = 2
n = 7 -> Minimum Steps = 3
n = 10 -> Minimum Steps = 3
n = 15 -> Minimum Steps = 4
n = 100 -> Minimum Steps = 7
```

ANALYSIS

We used Bottom-up memoization approach, it uses recursion with memoization to save answers and to avoid repeating calculations. It gives the minimum steps of n=1 -> 0, n=4 -> 2, n=7 -> 3, and n=10 -> 3. Without the memoization, it would result in a much longer runtime.

Bottom-Up Dynamic Programming Implementation

Welcome | ILOA1.cpp | ILOA2.cpp

ILOA2.cpp > main()

```cpp
// Activity 12 - ILO A: Minimum Steps to One (Bottom-Up DP)
#include <iostream>
#include <vector>
#include <algorithm>

int getMinSteps ( int n ){
    int dp[n+1], i;
    dp[1] = 0; // trivial case
    for( i = 2 ; i <= n ; i ++ ){
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = std::min( dp[i] , 1+ dp[i/2] );
        if(i%3==0) dp[i] = std::min( dp[i] , 1+ dp[i/3] );
    }
    return dp[n];
}

int main(){
    int tests[] = {1, 4, 7, 10, 15, 100};
    for(int n: tests)
        std::cout << "n = " << n << " -> Minimum Steps = " <<
    return 0;
}
```

Problems | Output | Debug Console | Terminal | Ports

```
PS D:\_DOWNLOADS\DO_THIS\HOA 12> cd "d:\_DOWNLOADS\DO_THIS\H
n = 1 -> Minimum Steps = 0
n = 4 -> Minimum Steps = 2
n = 7 -> Minimum Steps = 3
n = 10 -> Minimum Steps = 3
n = 15 -> Minimum Steps = 4
n = 100 -> Minimum Steps = 7
```

ANALYSIS

The program uses bottom-up, filling dp[] from 2 to n to find the minimum steps to reach 1. The time and space

complexity are both O(n). DP is efficient because it stores answers instead of recalculating them.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.